UNIT-V: Cloud and Edge Computing Models with IoT Use Cases

Syllabus:

- Introduction to Cloud Computing and Cloud Storage Models
- Edge and Fog Computing Concepts for IoT
- Web Servers and Cloud Platforms for IoT (AWS IoT, Azure IoT, etc.)
- Sensor-Cloud Architecture
- Data Analytics and Visualization Techniques for IoT Data
- IoT Use Cases: Smart Cities, Smart Homes, Connected Vehicles, Smart Grid, Industrial IoT
- Case Studies: Agriculture, Healthcare, Activity Monitoring

1. Introduction to Cloud Computing and Cloud Storage Models

1.1 What is Cloud Computing?

Cloud computing is the on-demand delivery of computing resources such as servers, storage, databases, networking, software and analytics over the Internet on a pay-as-you-go basis. Instead of buying and maintaining physical hardware, users rent resources from a cloud provider such as AWS, Microsoft Azure or Google Cloud.

According to **NIST**, cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned with minimal management effort.

Why cloud for IoT? IoT devices (sensors, actuators) have limited memory, power and processing capacity but generate huge volumes of data. The cloud offers the storage, processing power and scalability needed to collect, store, analyse and act on this data.

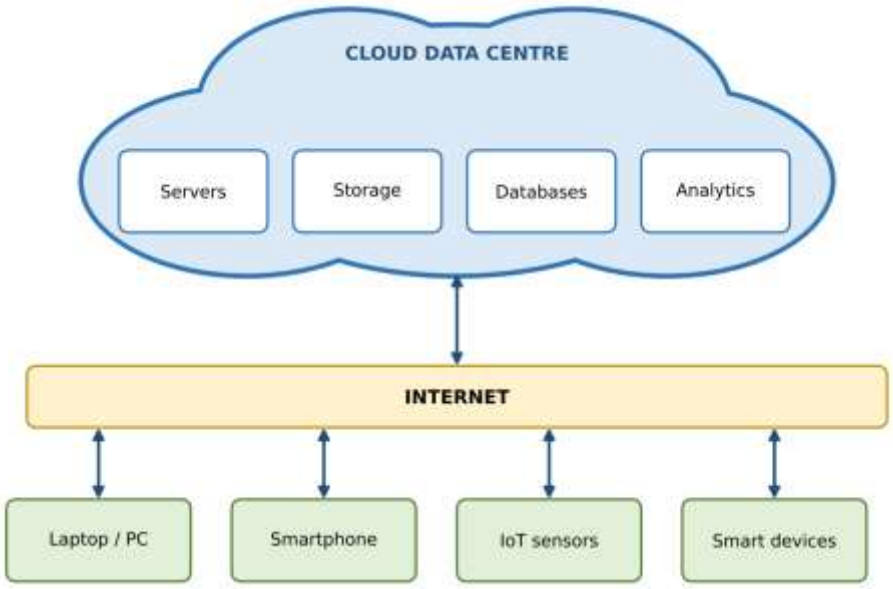


Figure 1.1: Overview of Cloud Computing (users, Internet, cloud data centre)

1.2 Essential Characteristics (NIST)

- **On-demand self-service** – users provision resources automatically without human interaction with the provider.
- **Broad network access** – services are available over the network from phones, laptops and IoT gateways.
- **Resource pooling** – provider resources are shared among many tenants (multi-tenancy).
- **Rapid elasticity** – resources scale up or down quickly as load changes.
- **Measured service** – usage is metered, so users pay only for what they use.

1.3 Cloud Service Models

Model	Full form / What is provided	Who uses it	Examples
IaaS	Infrastructure as a Service – virtual machines, storage, networks	System administrators, IT teams	AWS EC2, Azure Virtual Machines, Google Compute Engine
PaaS	Platform as a Service – runtime, databases, middleware, development tools	Developers	AWS Elastic Beanstalk, Azure App Service, Google App Engine
SaaS	Software as a Service – ready-to-use application over the web	End users	Gmail, Google Docs, Office 365, Salesforce

In IoT, an additional model is often mentioned: **IoT Platform as a Service** (for example AWS IoT Core, Azure IoT Hub), which provides device connectivity, management and data ingestion as a managed service.

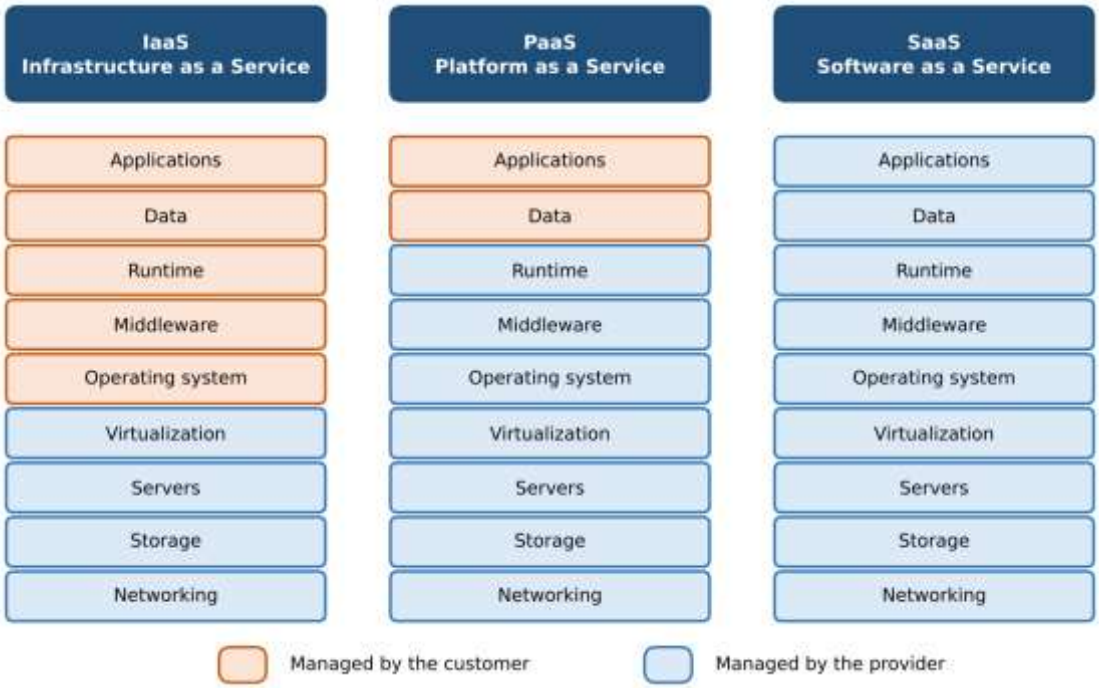


Figure 1.2: Cloud Service Models – IaaS, PaaS and SaaS Stack

1.4 Cloud Deployment Models

- **Public cloud** – owned and operated by a third-party provider and shared by many customers (AWS, Azure, GCP). Low cost, highly scalable, but less control.
- **Private cloud** – used exclusively by one organisation, hosted on-premises or by a provider. More security and control but higher cost.
- **Hybrid cloud** – combination of public and private clouds; sensitive data stays private while heavy computation uses the public cloud.
- **Community cloud** – shared by organisations with common concerns (e.g., government departments, universities).

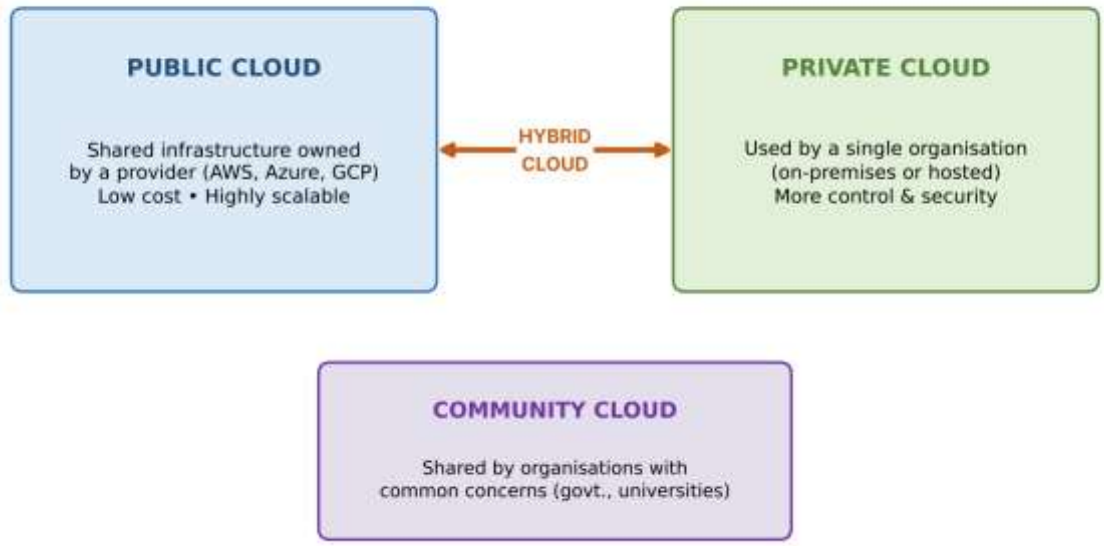


Figure 1.3: Cloud Deployment Models – Public, Private, Hybrid and Community

1.5 Cloud Storage Models

IoT systems produce a mix of structured, semi-structured and unstructured data. Cloud providers offer different storage models to suit each type.

Storage type	Description	Typical IoT use	Examples
Object storage	Data stored as objects (data + metadata + unique ID) in flat buckets; highly scalable and cheap	Raw sensor logs, images, video, backups, data lakes	Amazon S3, Azure Blob Storage, Google Cloud Storage
Block storage	Data stored in fixed-size blocks attached to a VM like a hard disk; low latency	Databases and applications that need fast read/write	Amazon EBS, Azure Disk
File storage	Hierarchical files and folders shared over a network (NFS/SMB)	Shared configuration files, firmware images	Amazon EFS, Azure Files
Database storage (SQL)	Structured tables with a fixed schema	Device registry, user accounts, billing	Amazon RDS, Azure SQL, MySQL

Storage type	Description	Typical IoT use	Examples
Database storage (NoSQL / Time-series)	Flexible schema; time-stamped data optimised for fast writes	Continuous sensor readings, telemetry	DynamoDB, MongoDB, InfluxDB, Azure Cosmos DB, Amazon Timestream

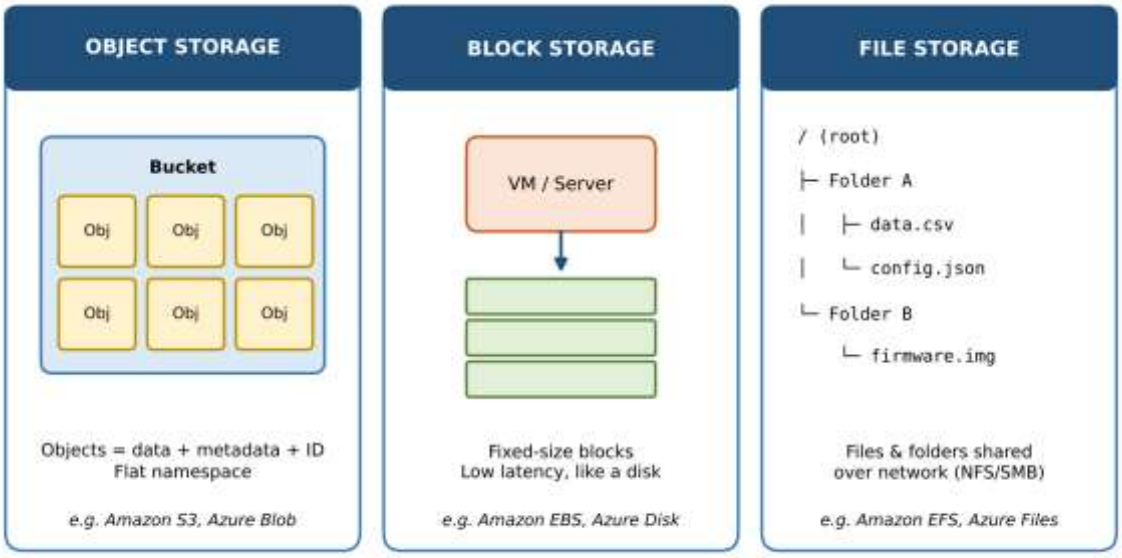


Figure 1.4: Types of Cloud Storage – Object, Block and File Storage

Advantages of cloud for IoT

- Virtually unlimited storage and compute; scales with the number of devices.
- Pay-as-you-go lowers upfront cost.
- High availability and disaster recovery through replication.
- Easy remote access, device management and global reach.

Limitations

- Latency due to distance between device and data centre.
- Dependency on Internet connectivity.
- Bandwidth cost for sending large raw data.
- Data privacy, security and vendor lock-in concerns.

These limitations are the main reason **edge and fog computing** were introduced (next section).

2. Edge and Fog Computing Concepts for IoT

2.1 Need for Edge and Fog Computing

Sending all IoT data to a distant cloud has drawbacks. Billions of devices produce zettabytes of data; time-critical applications (self-driving cars, industrial safety systems, remote surgery) cannot tolerate cloud round-trip delays; and continuous uploading wastes bandwidth and exposes private data.

- **Latency** – decisions are needed in milliseconds.
- **Bandwidth** – filtering data locally reduces network load and cost.
- **Reliability** – the system should keep working even if the Internet link fails.
- **Privacy and security** – sensitive data can be processed locally.
- **Cost** – less data transferred and stored in the cloud.

2.2 Edge Computing

Edge computing means processing data at or very near the source where it is generated – on the IoT device itself, on a smart sensor, or on a nearby gateway – rather than in a centralised cloud. The "edge" is the boundary between the physical world and the network.

- **Edge devices:** sensors, actuators, smart cameras, microcontrollers such as Raspberry Pi, Arduino, ESP32, NVIDIA Jetson.
- **Edge gateways:** collect data from several devices, translate protocols (e.g., Zigbee/BLE to MQTT), filter and aggregate data, and forward it to the cloud.
- **Typical edge tasks:** data filtering, aggregation, local analytics, real-time control, running small ML models (TinyML).

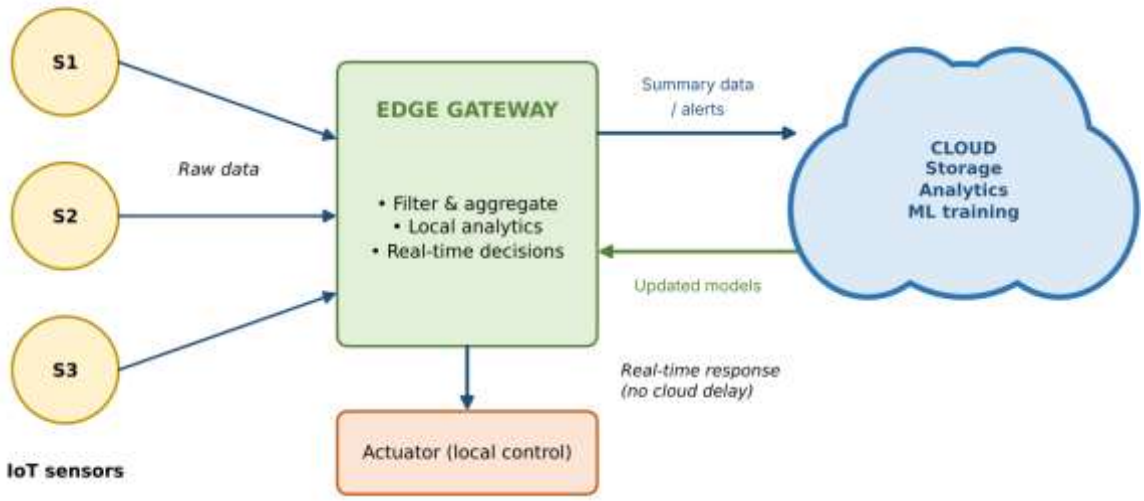


Figure 2.1: Edge Computing in an IoT System

2.3 Fog Computing

Fog computing (term coined by Cisco) is a decentralised architecture that places computing, storage and networking services **between the edge devices and the cloud**. Fog nodes include routers,

switches, base stations, industrial controllers and micro data centres. Fog extends the cloud closer to the things that produce data, and typically covers a larger area than a single edge device (e.g., a factory floor or a city block).

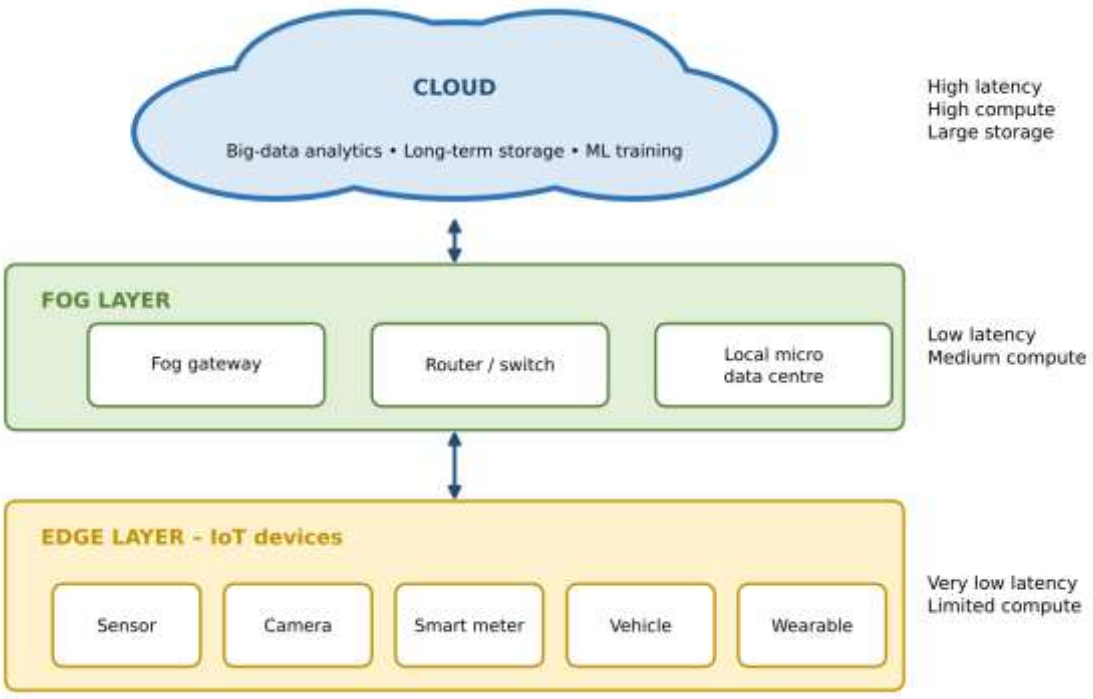


Figure 2.2: Three-Tier Architecture: Edge – Fog – Cloud

2.4 Cloud vs Fog vs Edge Computing

Parameter	Cloud	Fog	Edge
Location of processing	Remote centralised data centres	Local network (gateways, routers, local servers)	On the device or its immediate gateway
Latency	High (100 ms or more)	Low (about 10–50 ms)	Very low (about 1–10 ms)
Computing power	Very high	Medium	Low to limited
Storage	Very large	Moderate	Small
Bandwidth use	High	Medium	Low
Scope	Global	Regional / local area	Single device or site
Best for	Big-data analytics, long-term storage, ML training	Coordinating many local devices, real-time analytics	Instant response, offline operation

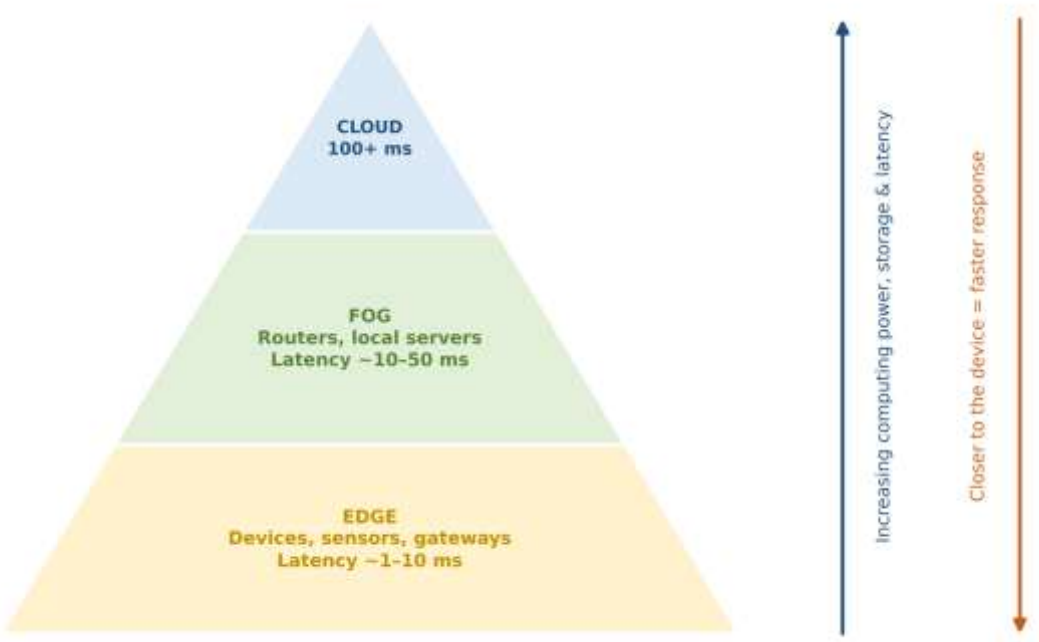


Figure 2.3: Comparison of Cloud, Fog and Edge Computing (latency vs. computing power)

2.5 Advantages and Challenges

Advantages

- Ultra-low latency and real-time response.
- Reduced network traffic and cloud costs.
- Works with intermittent connectivity.
- Better data privacy and local compliance.
- Improved scalability and reliability.

Challenges

- Limited resources on edge/fog nodes.
- Managing and securing a large number of distributed nodes.
- Lack of standardisation across vendors.
- Software and firmware updates at scale.
- Task offloading decisions – what to run at edge, fog or cloud.

2.6 Examples

- A smart camera detecting a fall or intruder locally and sending only an alert to the cloud.
- An autonomous vehicle processing sensor data on board for braking decisions.
- A factory gateway analysing vibration data to stop a machine immediately and sending summaries to the cloud.
- **AWS IoT Greengrass** and **Azure IoT Edge** – run cloud logic and ML models on local devices.

3. Web Servers and Cloud Platforms for IoT

3.1 Web Servers for IoT

A **web server** is software (running on hardware) that accepts requests from clients over HTTP/HTTPS and returns responses such as web pages, JSON data or files. In IoT, web servers act as the interface between devices, databases and users.

Roles of a web server in IoT

- Receive sensor data sent by devices through **HTTP POST** or REST APIs.
- Serve dashboards and web/mobile applications to users.
- Send commands to actuators (e.g., switch a light ON/OFF).
- Authenticate users and devices and log activity.

Embedded vs. external web servers

- **Embedded web server:** lightweight server running directly on the device (ESP8266/ESP32, Raspberry Pi) to provide local control pages over Wi-Fi.
- **External/cloud web server:** Apache, Nginx, Node.js (Express) or Flask/Django applications hosted on cloud VMs and handling many devices.

Common communication protocols

Protocol	Model	Features	Use in IoT
HTTP/HTTPS (REST)	Request–response	Simple, universal, but heavy headers	Web dashboards, APIs, ThingSpeak updates
MQTT	Publish–subscribe via broker	Lightweight, low bandwidth, QoS levels 0/1/2	Telemetry from constrained devices
CoAP	Request–response over UDP	Very light, designed for constrained devices	Low-power sensor networks
WebSocket	Full-duplex	Persistent two-way connection	Real-time dashboards

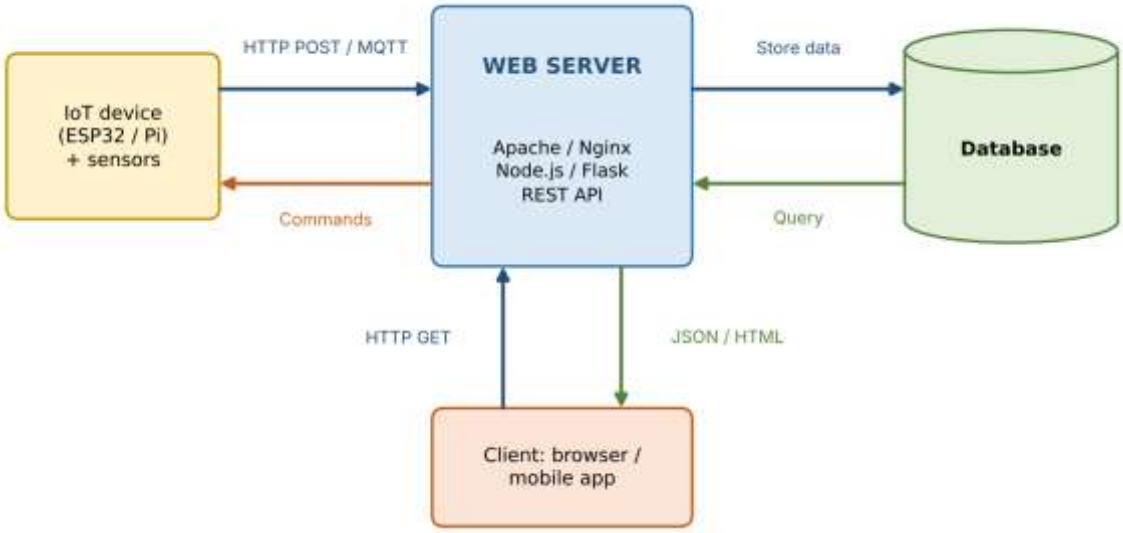


Figure 3.1: Interaction between IoT Device, Web Server, Database and Client

3.2 Cloud Platforms for IoT

An **IoT cloud platform** provides ready-made services for device connectivity, device management, security, data storage, rule processing, analytics and application development, so developers do not need to build everything from scratch.

A. AWS IoT (Amazon Web Services)

AWS IoT Core lets connected devices interact securely with cloud applications and other devices. Devices connect using MQTT, HTTPS or WebSockets.

- **Device Gateway** – entry point for devices; supports MQTT, HTTPS, WebSockets and scales automatically.
- **Message Broker** – publish/subscribe broker routing messages between devices and applications.
- **Device Registry** – stores the identity and attributes of every device (“thing”).
- **Device Shadow** – a JSON document holding a device’s last reported and desired state, so apps can interact even when the device is offline.
- **Rules Engine** – SQL-like rules that filter and route data to other AWS services such as S3, DynamoDB, Lambda, Kinesis or SNS.
- **Security** – X.509 certificates, policies, AWS IAM, TLS encryption; **Device Defender** monitors for anomalies.
- **Related services:** IoT Greengrass (edge), IoT Analytics, IoT SiteWise (industrial), IoT Events, IoT Device Management, Amazon QuickSight (visualisation).

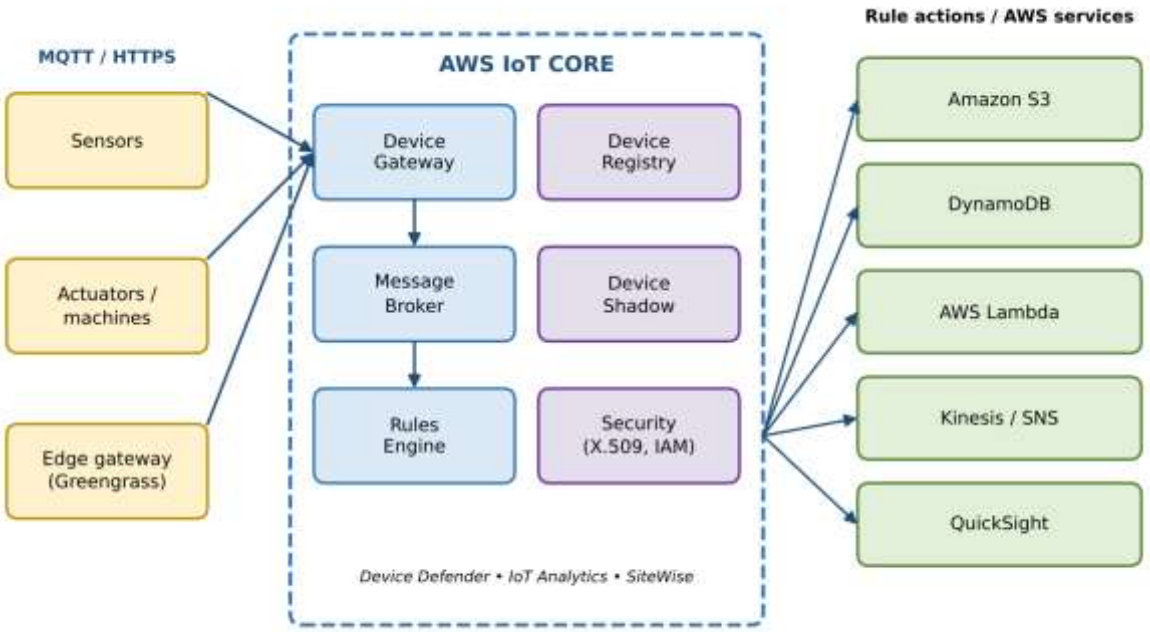


Figure 3.2: AWS IoT Core Architecture

B. Microsoft Azure IoT

- **Azure IoT Hub** – managed hub for bi-directional communication between devices and the cloud; supports MQTT, AMQP and HTTPS; provides device-to-cloud telemetry, cloud-to-device commands and **device twins**.
- **Device Provisioning Service (DPS)** – zero-touch automatic registration of devices to the right hub.
- **Azure IoT Edge** – deploy containerised cloud workloads, AI models and analytics on edge devices.
- **Azure Stream Analytics** – real-time processing of streaming data using SQL-like queries.
- **Azure Digital Twins** – models real-world environments such as buildings or factories as digital twins.
- **Azure IoT Central** – ready-made SaaS solution for quickly building IoT applications.
- **Storage, analytics and visualisation:** Blob Storage, Cosmos DB, Azure Data Explorer, Azure Machine Learning, Power BI.

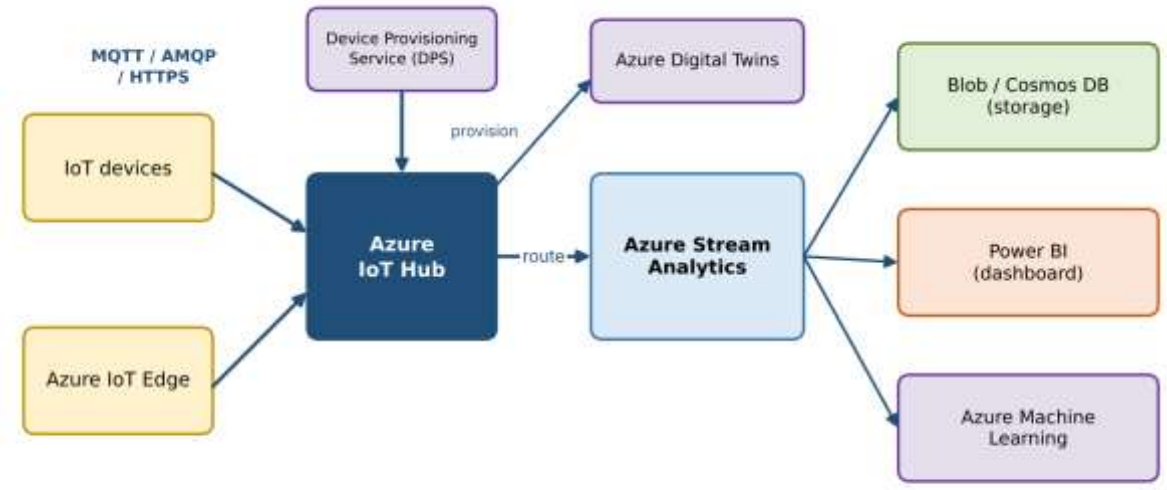


Figure 3.3: Azure IoT Reference Architecture (IoT Hub, Stream Analytics, Storage, Power BI)

C. Other platforms

- **Google Cloud:** Google Cloud IoT Core was retired in August 2023; Google now recommends partner platforms with Pub/Sub, Dataflow and BigQuery for IoT data pipelines.
- **IBM Watson IoT Platform** – device management and AI-based analytics.
- **ThingSpeak** (MathWorks) – simple open platform for collecting, analysing and plotting sensor data using MATLAB; ideal for student projects.
- **Others:** Oracle IoT, ThingsBoard (open source), Blynk, Particle, Cisco Kinetic.

3.3 Comparison of AWS IoT and Azure IoT

Feature	AWS IoT Core	Azure IoT Hub
Protocols	MQTT, HTTPS, WebSockets, LoRaWAN	MQTT, AMQP, HTTPS, WebSockets
Device state	Device Shadow	Device Twin
Data routing	Rules Engine	Message routing / Event Grid
Edge computing	IoT Greengrass	Azure IoT Edge
Auto-provisioning	Fleet Provisioning	Device Provisioning Service
Stream analytics	Kinesis, IoT Analytics	Stream Analytics
Visualisation	QuickSight, Grafana	Power BI

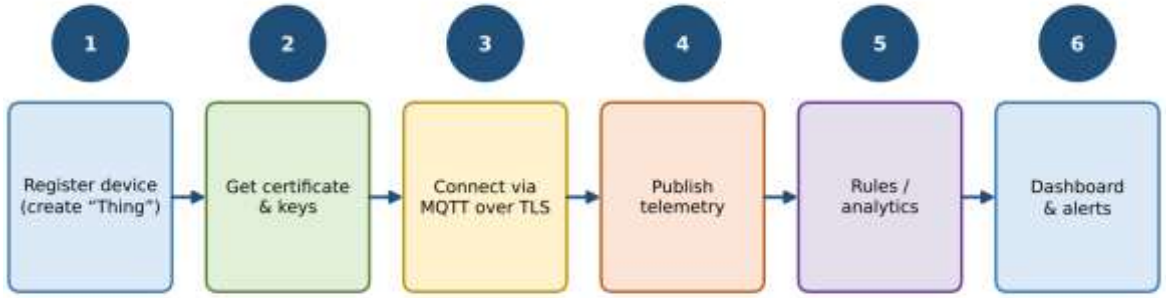


Figure 3.4: Typical Workflow of Connecting a Device to an IoT Cloud Platform

4. Sensor-Cloud Architecture

4.1 Concept

The **Sensor-Cloud** is an infrastructure that integrates wireless sensor networks (WSNs) with cloud computing. Sensor data from many physical sensors owned by different organisations is collected, stored and processed in the cloud, and the sensors are **virtualised** so that users can use them as services – often called **Sensing as a Service (S2aaS)**.

Sensor virtualisation: one or more physical sensors are abstracted into a *virtual sensor* that behaves like a dedicated sensor to the user. Many users can share the same physical sensor, and a user can combine multiple sensors into one virtual sensor without worrying about hardware details.

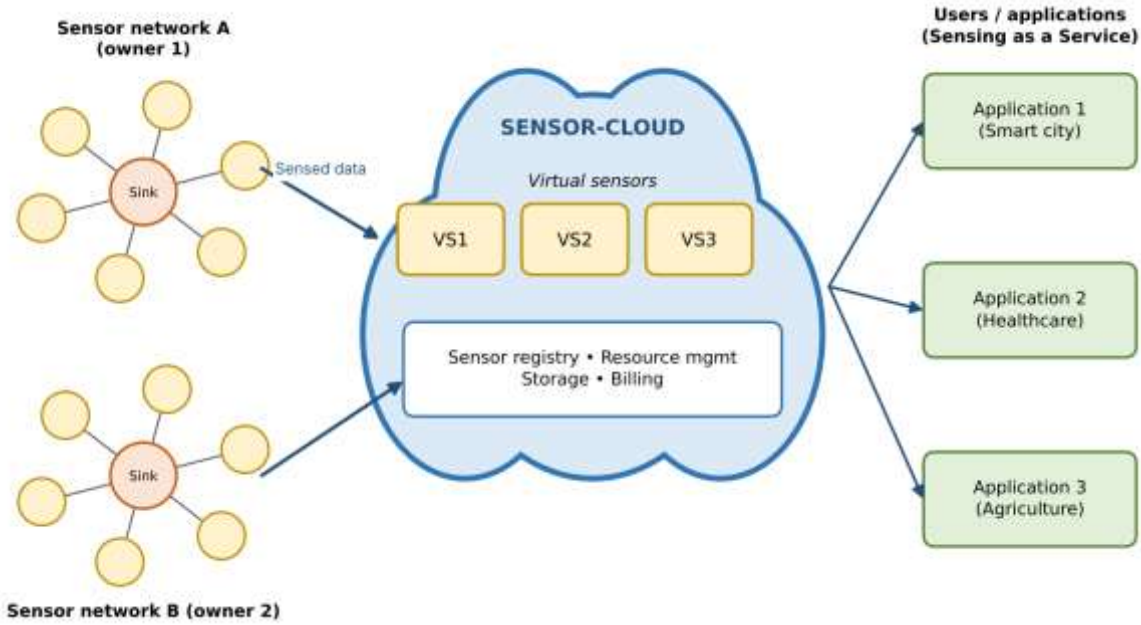


Figure 4.1: Concept of Sensor-Cloud (physical sensors, virtual sensors and users)

4.2 Layered Architecture

Layer	Function
Sensor layer (physical layer)	Physical sensors and actuators of WSNs measure temperature, humidity, motion, etc.
Sensor gateway / network layer	Sink nodes and gateways collect data and send it to the cloud using Wi-Fi, cellular, LoRa, Zigbee, etc.
Cloud / virtualisation layer	Sensor registry, virtual sensor creation, resource management, data storage and processing, billing and SLA handling
Application layer	Applications and users access sensor data and services through APIs, portals and dashboards

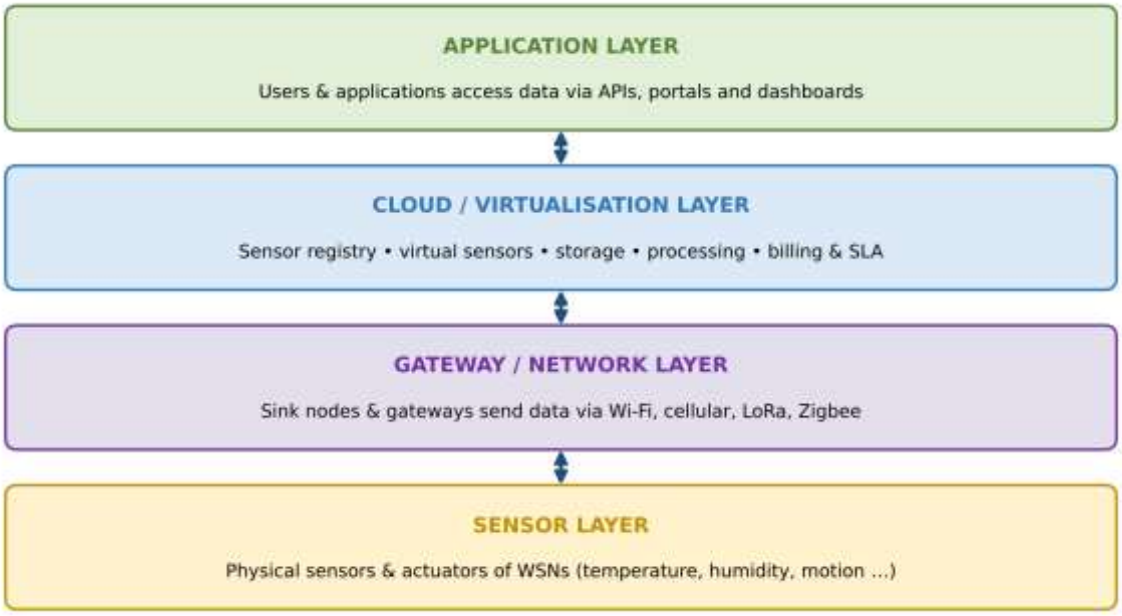


Figure 4.2: Layered Architecture of Sensor-Cloud

4.3 Key Components

- **Sensor owners** – provide physical sensors to the infrastructure.
- **Sensor-cloud service provider** – manages the cloud, virtualisation and billing.
- **End users** – request sensing services without owning sensors.
- **Sensor provisioning and management module** – registers, monitors and allocates sensors.
- **Data storage and analytics module** – stores and processes the sensed data.

4.4 Sensor-Cloud vs. Traditional WSN

Traditional WSN	Sensor-Cloud
Sensors dedicated to one application/owner	Sensors shared by many applications through virtualisation
Limited storage and processing	Almost unlimited storage and processing in the cloud

Traditional WSN	Sensor-Cloud
Low resource utilisation	High utilisation and cost sharing
Difficult to scale	Easily scalable
Application-specific	Generic, service-oriented

4.5 Advantages and Challenges

- **Advantages:** resource sharing, scalability, pay-per-use sensing, easy data access, big-data analytics, multi-application support.
- **Challenges:** data security and privacy, sensor energy consumption, network delay, standardisation, QoS and SLA management, data quality and trust.

5. Data Analytics and Visualization Techniques for IoT Data

5.1 Characteristics of IoT Data

- **Volume** – huge amount of data from billions of devices.
- **Velocity** – generated continuously, often in real time.
- **Variety** – numbers, text, images, audio, video, logs.
- **Veracity** – data may be noisy, missing or inaccurate.
- **Time-series nature** – every reading has a timestamp.

5.2 IoT Data Analytics Pipeline

The data goes through the following stages:

- **Data collection** – sensors and devices generate data.
- **Data ingestion** – transported via MQTT/HTTP to a broker or message queue (AWS IoT Core, Azure IoT Hub, Apache Kafka).
- **Pre-processing** – cleaning, removing noise and outliers, filling missing values, normalisation, aggregation.
- **Storage** – data lake (S3, Blob), time-series DB (InfluxDB), NoSQL (MongoDB, DynamoDB), data warehouse.
- **Processing and analysis** – batch or stream processing and machine learning.
- **Visualisation and action** – dashboards, alerts and automatic control of actuators.

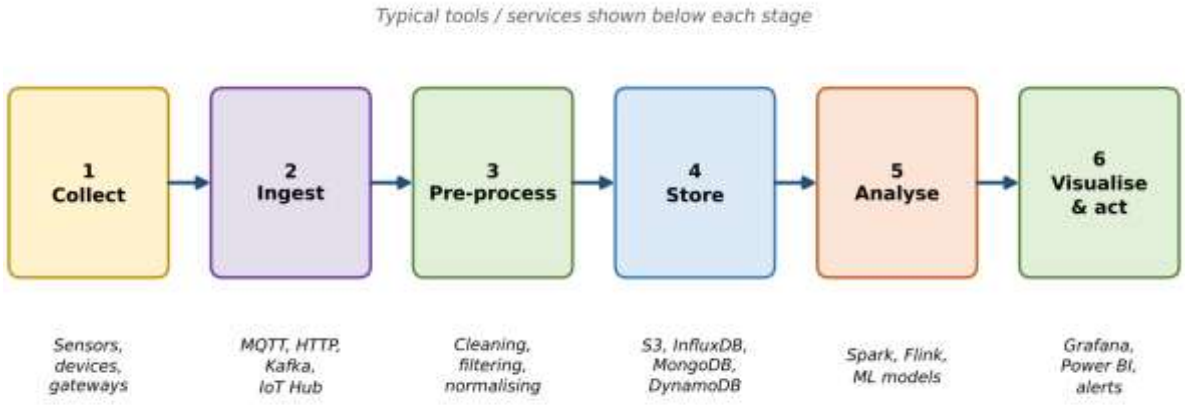


Figure 5.1: IoT Data Analytics Pipeline (Collect → Ingest → Pre-process → Store → Analyse → Visualise)

5.3 Processing Approaches

Aspect	Batch processing	Stream (real-time) processing
Data handled	Large stored data sets processed at intervals	Continuous data processed as it arrives
Latency	Minutes to hours	Milliseconds to seconds
Use	Trend analysis, reports, ML model training	Alarms, anomaly detection, live monitoring
Tools	Hadoop MapReduce, Apache Spark	Apache Kafka, Spark Streaming, Apache Flink, AWS Kinesis, Azure Stream Analytics

5.4 Types of Analytics

Type	Question answered	IoT example
Descriptive	What happened?	Daily average temperature of a cold-storage unit
Diagnostic	Why did it happen?	Finding why a machine overheated using sensor logs
Predictive	What will happen?	Predicting machine failure (predictive maintenance)
Prescriptive	What should be done?	Recommending the best irrigation schedule or automatically shutting down a machine

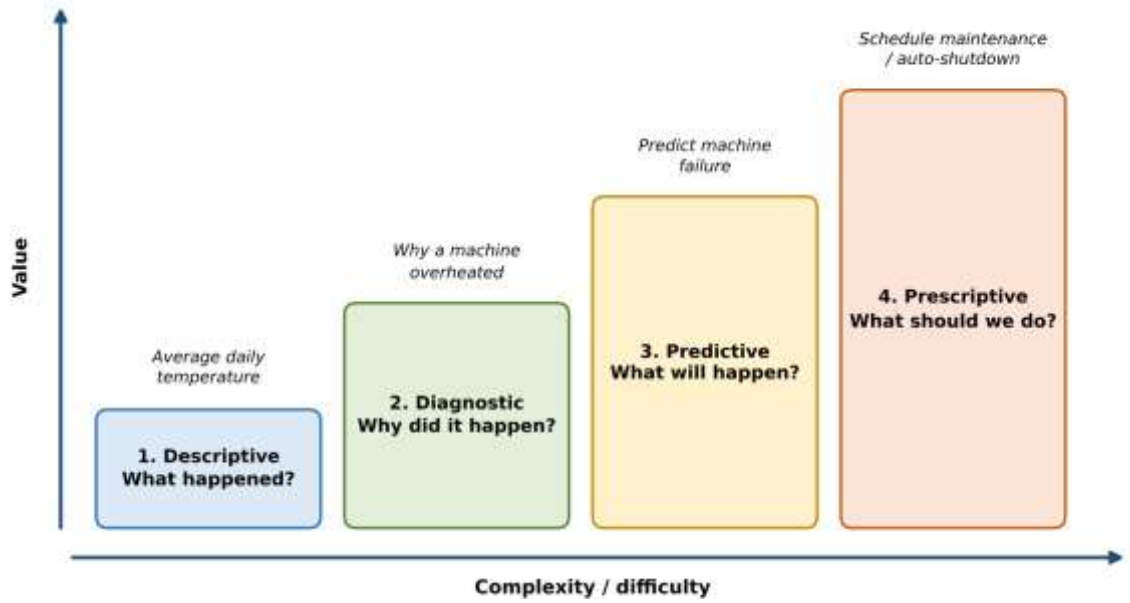


Figure 5.2: Four Types of Data Analytics (Descriptive, Diagnostic, Predictive, Prescriptive)

5.5 Analytics Techniques and Tools

- **Statistical analysis:** mean, variance, moving average, correlation, regression.
- **Machine learning:** classification (activity recognition), regression (energy forecasting), clustering (grouping similar devices), anomaly detection (fault or intrusion detection).
- **Deep learning:** CNNs for image/video from cameras; LSTM/RNNs for time-series prediction.
- **Edge analytics:** lightweight models (TinyML, TensorFlow Lite) running on devices.
- **Tools:** Python (Pandas, NumPy, scikit-learn, TensorFlow), MATLAB, R, Apache Spark MLlib, Amazon SageMaker, Azure Machine Learning.

5.6 Visualization Techniques

Data visualisation converts raw data into graphical forms so that humans can quickly understand trends, patterns and anomalies.

Technique	Best used for
Line chart / time-series plot	Showing sensor values (temperature, voltage) over time
Bar chart / histogram	Comparing categories or showing value distribution
Pie / donut chart	Showing proportions (e.g., energy consumption by appliance)
Gauge / meter	Displaying the current value of a single parameter (speed, tank level)
Scatter plot	Showing relationship between two variables
Heat map	Showing intensity across space/time (traffic density, temperature map)
Geo-map / GIS map	Tracking vehicles, assets and city sensors by location
Dashboards and alerts	Combining many widgets for real-time monitoring with threshold notifications

- **Visualisation tools:** Grafana, Kibana (ELK stack), Tableau, Microsoft Power BI, Amazon QuickSight, ThingSpeak charts, Node-RED dashboard, Matplotlib/Plotly in Python.

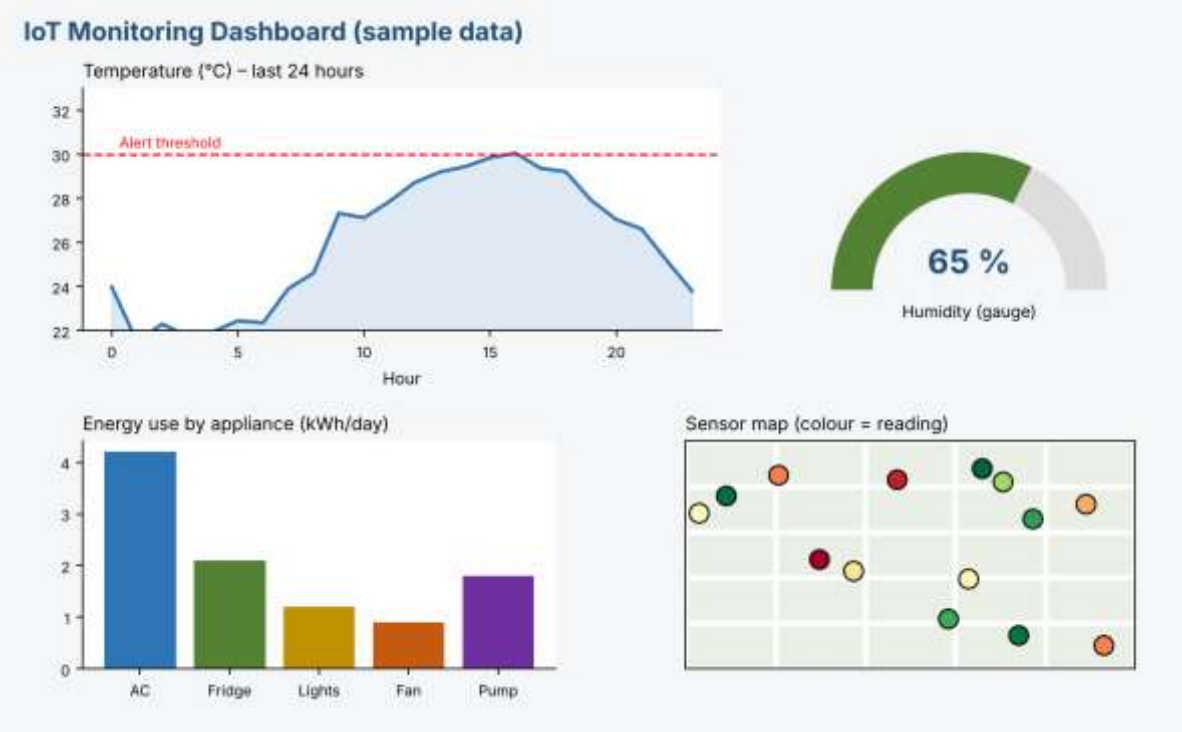


Figure 5.3: Sample IoT Dashboard (line chart, gauge, bar chart and map)

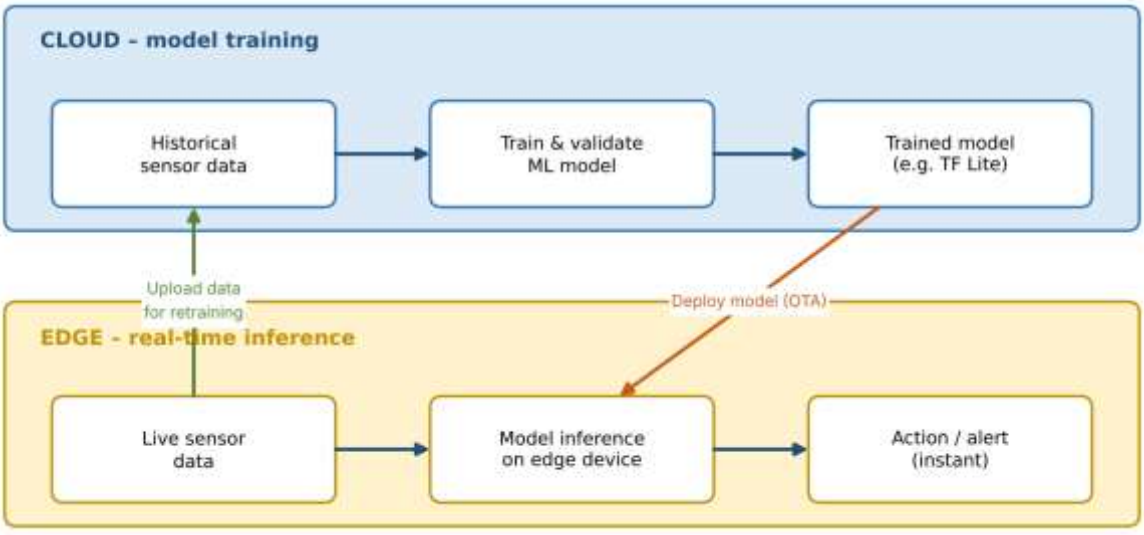


Figure 5.4: Machine Learning Workflow for IoT Data (training in cloud, deployment at edge)

6. IoT Use Cases

6.1 Smart Cities

A **smart city** uses IoT sensors, communication networks and data analytics to improve urban services, sustainability and quality of life.

- **Smart parking:** sensors in parking bays detect vacant slots and guide drivers via an app.
- **Smart street lighting:** lights dim or brighten based on motion and daylight, saving energy.
- **Traffic management:** cameras and road sensors adjust signal timings and reduce congestion.
- **Waste management:** ultrasonic sensors in bins signal when they are full to optimise collection routes.
- **Water management:** leakage detection and quality monitoring.
- **Air-quality and noise monitoring, public safety (CCTV analytics), emergency response.**

Technologies: LoRaWAN, NB-IoT, 5G, edge gateways, city-wide cloud platform and dashboards. **Example:** Smart Cities Mission in India, Barcelona and Singapore smart-city projects.

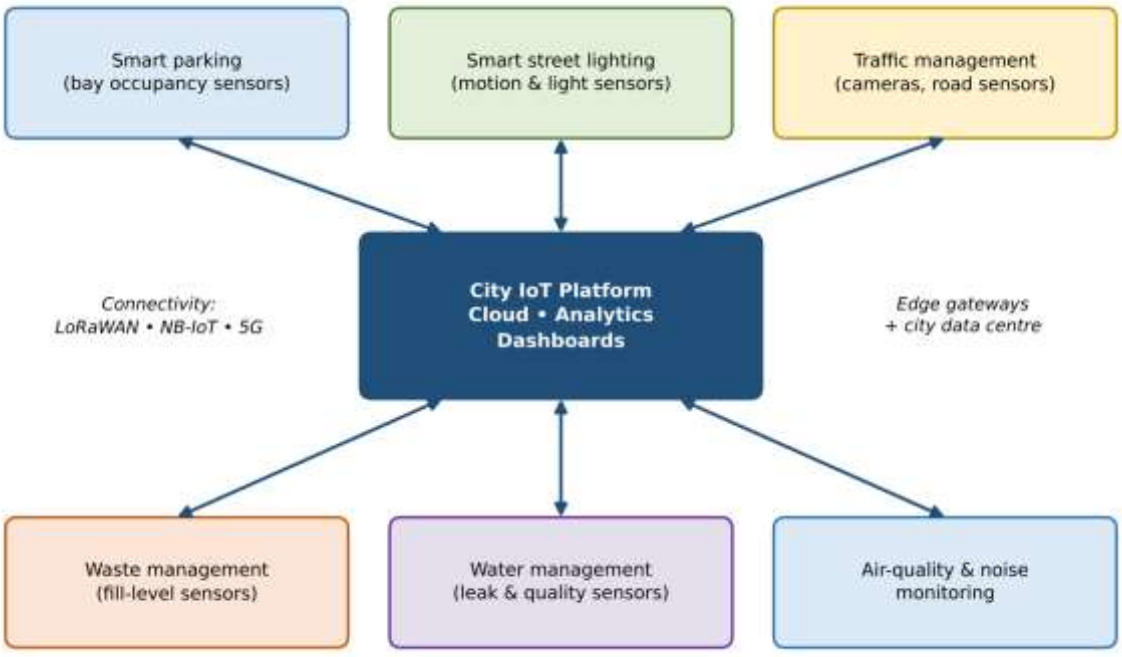


Figure 6.1: Smart City Architecture with IoT Applications

6.2 Smart Homes

A **smart home** connects household appliances and systems so that they can be monitored and controlled automatically or remotely.

- **Home automation:** smart lights, fans, switches, thermostats (e.g., Nest), smart plugs.
- **Security:** smart door locks, video doorbells, motion sensors, CCTV, alarms.
- **Energy management:** smart meters and appliances that reduce power use.
- **Voice assistants:** Amazon Alexa, Google Assistant for voice control.

- **Safety:** smoke, gas-leak and water-leak sensors with instant alerts.

Technologies: Wi-Fi, Zigbee, Z-Wave, Bluetooth LE, Thread/Matter; a home hub or gateway; cloud for remote access through a mobile app.

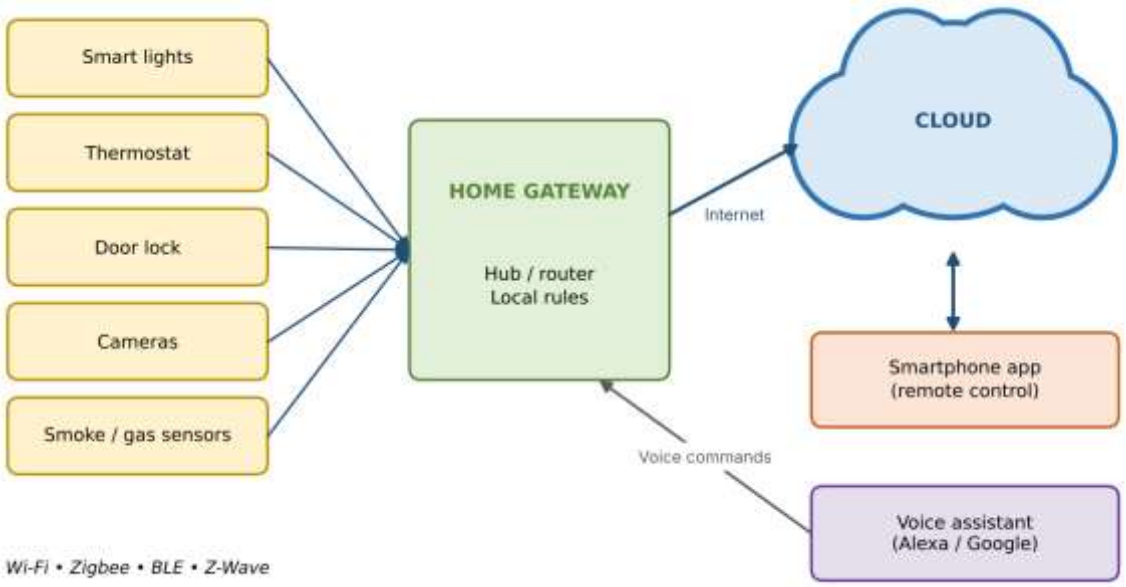


Figure 6.2: Smart Home System Architecture

6.3 Connected Vehicles

Connected vehicles use Internet and wireless communication to exchange data with other vehicles, road infrastructure, pedestrians and the cloud. This is called **V2X (Vehicle-to-Everything)**.

- **V2V** – Vehicle to Vehicle: share speed and position to avoid collisions.
- **V2I** – Vehicle to Infrastructure: traffic lights, toll booths, road sensors.
- **V2P** – Vehicle to Pedestrian; **V2N** – Vehicle to Network/Cloud.
- **Applications:** collision warning, adaptive cruise control, real-time navigation, fleet tracking, remote diagnostics and over-the-air (OTA) updates, usage-based insurance, emergency eCall, autonomous driving.
- **Technologies:** GPS, CAN bus, OBD-II, DSRC, cellular (4G/5G C-V2X), cameras, radar, LiDAR, edge computing in the vehicle for instant decisions.

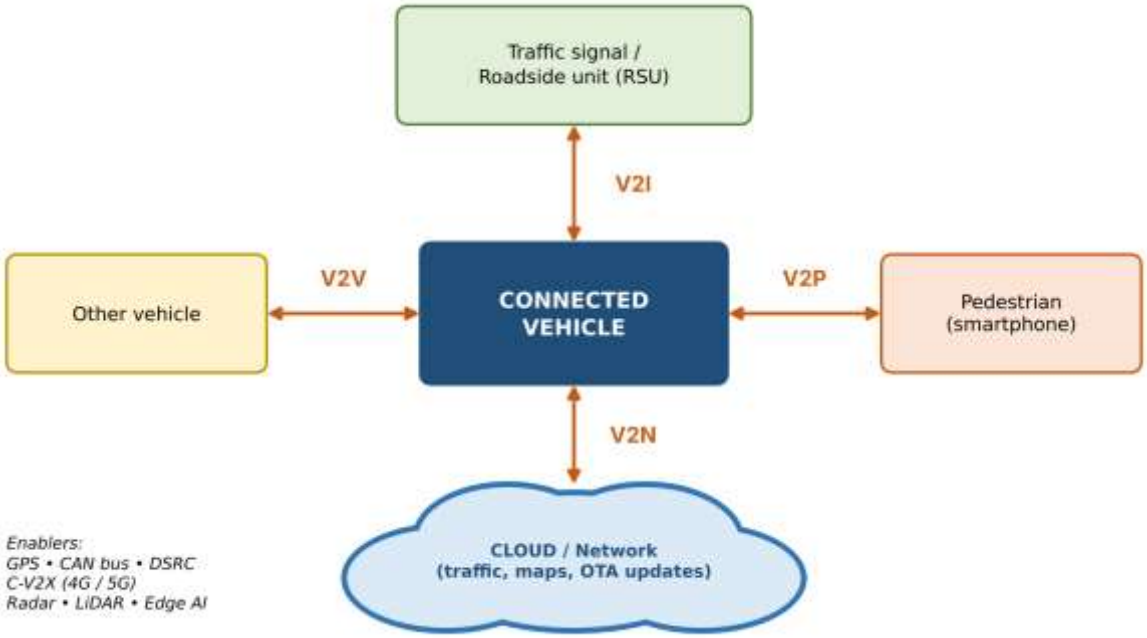


Figure 6.3: Connected Vehicle Ecosystem (V2V, V2I, V2P and V2N Communication)

6.4 Smart Grid

A **smart grid** is an electricity network that uses digital communication, sensors and control systems to monitor and manage the generation, transmission, distribution and consumption of electricity in a two-way flow of power and information.

- **Smart meters (AMI):** record consumption in real time and send readings to the utility.
- **Demand response:** consumers are encouraged to shift usage during peak hours via time-of-use pricing.
- **Fault detection and self-healing:** sensors detect faults and automatically isolate the affected section.
- **Integration of renewables:** solar and wind plus battery storage and electric vehicles.
- **Load forecasting and theft detection** using analytics.

Benefits: better reliability, fewer outages, reduced losses, lower costs, and integration of clean energy.
Challenges: cybersecurity, interoperability, high investment.

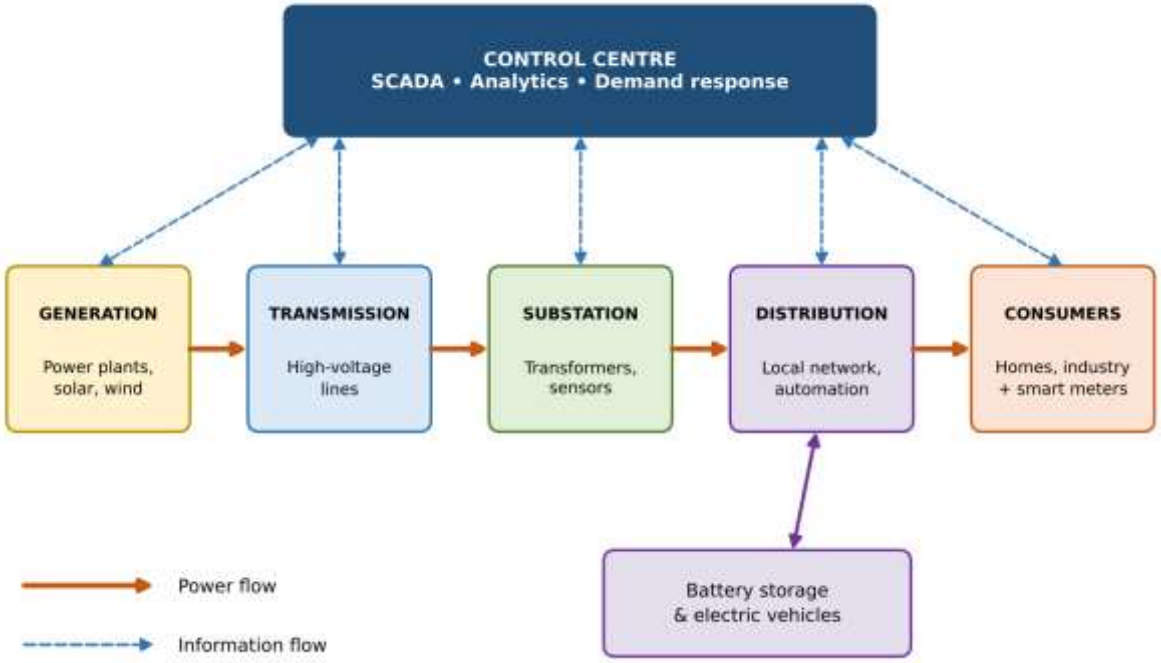


Figure 6.4: Smart Grid Architecture (Generation, Transmission, Distribution, Smart Meters, Control Centre)

6.5 Industrial IoT (IIoT)

Industrial IoT applies IoT in manufacturing, oil & gas, mining, logistics and energy to improve efficiency, safety and productivity. It is a key part of **Industry 4.0**.

- **Predictive maintenance:** vibration, temperature and current sensors predict machine failures before they occur, reducing downtime.
- **Asset tracking and supply chain:** RFID/GPS to track goods and equipment.
- **Process monitoring and quality control:** real-time monitoring and machine-vision inspection.
- **Worker safety:** wearables and gas/hazard sensors.
- **Digital twins:** virtual replicas of machines and plants for simulation and optimisation.

Technologies: industrial sensors, PLCs, SCADA, OPC-UA, MQTT, industrial Ethernet, 5G, edge gateways, cloud analytics (AWS IoT SiteWise, Azure IoT, Siemens MindSphere, GE Predix).

Aspect	Consumer IoT	Industrial IoT
Examples	Smart watch, smart bulb	Smart factory, power plant
Reliability need	Moderate	Very high (mission-critical)
Latency	Tolerant	Very low, deterministic
Scale	Home or individual	Enterprise or national
Security	Important	Critical (safety of people and plants)

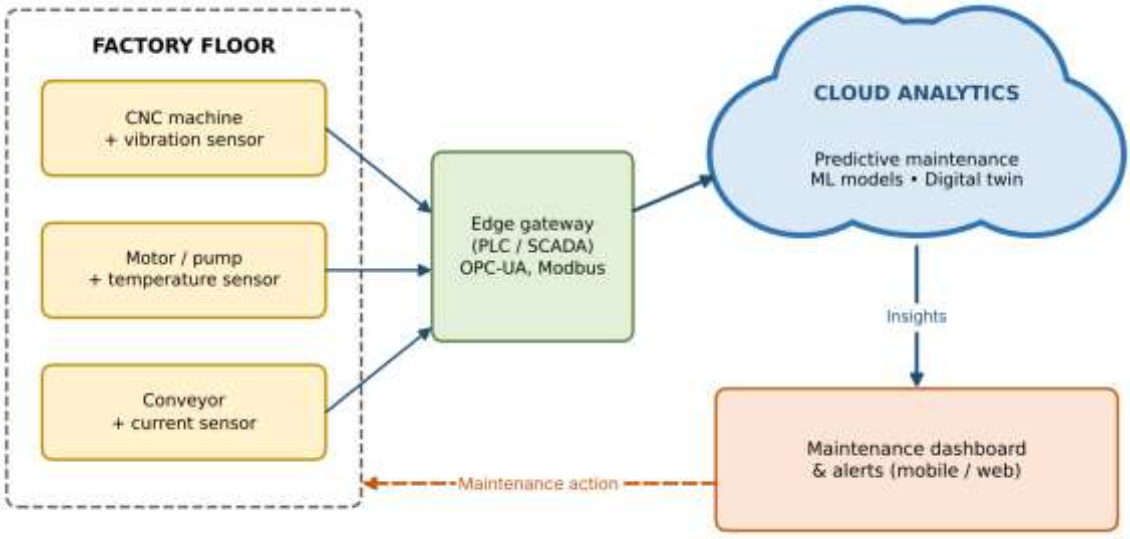


Figure 6.5: Industrial IoT Architecture (Machines, Edge Gateway, Cloud Analytics, Maintenance Dashboard)

7. Case Studies

7.1 Agriculture (Smart Farming / Precision Agriculture)

Problem: Traditional farming depends on guesswork, leading to water wastage, excess fertiliser use and crop loss. IoT enables data-driven farming.

System components

- **Sensors:** soil moisture, soil pH, temperature, humidity, rainfall, light, leaf wetness, water level.
- **Controller/edge node:** Arduino, ESP32 or Raspberry Pi reading sensors.
- **Communication:** LoRa, Zigbee, Wi-Fi or GSM/NB-IoT for remote fields.
- **Cloud and analytics:** data stored and analysed to decide irrigation and detect disease risk; farmers see data on a mobile app.
- **Actuators:** water pumps, solenoid valves, sprinklers, drip irrigation controllers.
- **Drones and satellite imaging:** crop health monitoring and spraying.

Working

- Soil-moisture sensor reads the soil condition continuously.
- If moisture drops below a threshold, the controller (or the cloud rule) switches the pump ON through a relay.
- Data is also sent to the cloud, where analytics combine it with weather forecasts for better scheduling.
- The farmer gets notifications and can control the system from a smartphone.

Benefits

- Saves water (up to 30–50 percent in many drip-irrigation systems) and fertiliser.
- Higher yield and quality; early disease and pest detection.

- Reduced labour and remote farm management.

Examples: Smart irrigation systems, livestock tracking, greenhouse monitoring, drone-based crop spraying, Microsoft FarmBeats.

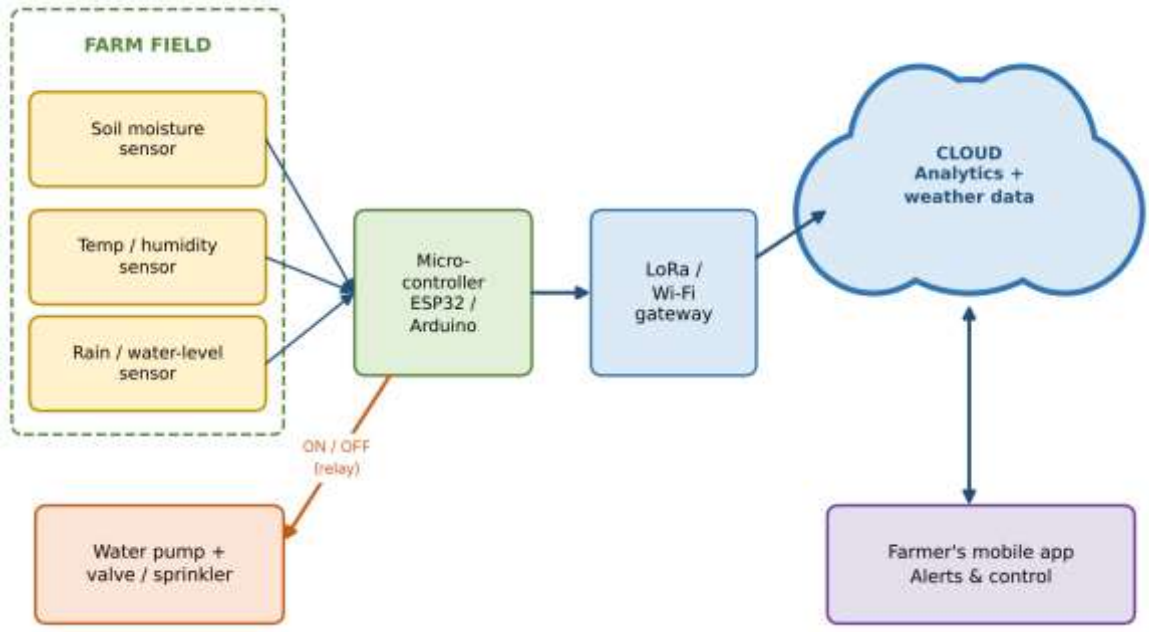


Figure 7.1: IoT-Based Smart Irrigation / Precision Agriculture System

7.2 Healthcare (Smart Healthcare / IoMT)

The **Internet of Medical Things (IoMT)** connects medical devices, wearables and hospital systems to give continuous, remote and personalised care.

System components

- **Wearable and body sensors:** heart-rate (PPG), ECG, SpO_2, blood pressure, glucose, temperature, respiration.
- **Gateway:** smartphone or home hub using Bluetooth LE to receive data.
- **Cloud platform:** stores Electronic Health Records (EHR), runs analytics and AI for anomaly detection.
- **Applications:** dashboards for doctors, mobile app for patients, alerts for caregivers and emergency services.

Applications

- **Remote Patient Monitoring (RPM)** for chronic conditions such as diabetes, hypertension and heart disease.
- **Smart pills and implantable devices**, smart insulin pumps and continuous glucose monitors.
- **Hospital asset and medicine tracking** with RFID; smart beds and ICU monitoring.
- **Elderly care and emergency alerts** such as fall detection.
- **Telemedicine** and teleconsultation.

Benefits and challenges

- **Benefits:** early detection, fewer hospital visits, lower cost, better patient outcomes, access in rural areas.
- **Challenges:** patient data privacy (HIPAA, GDPR, India's DPDP Act), device accuracy, security of medical devices, battery life, interoperability.

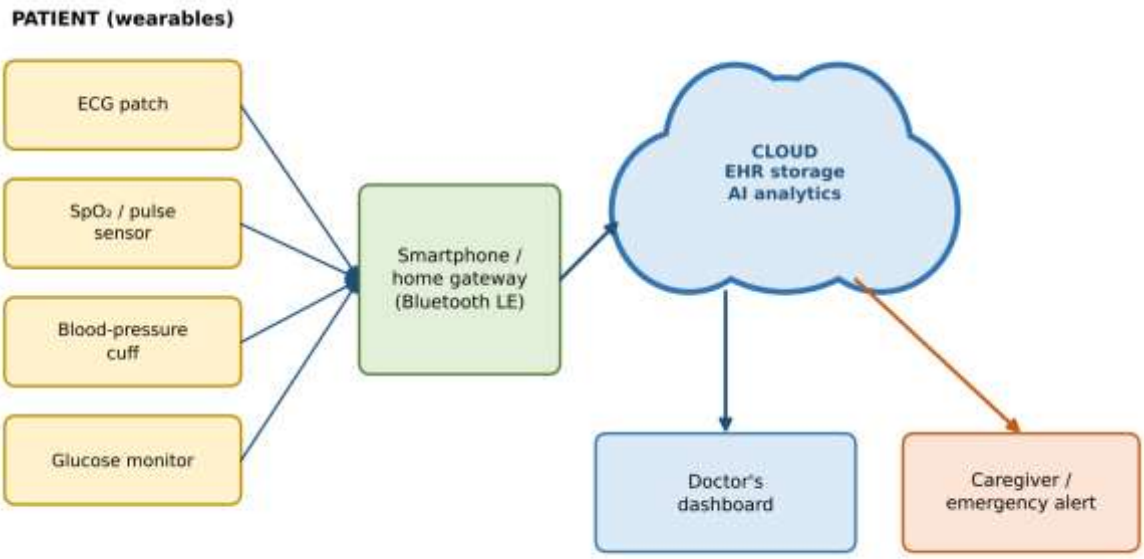


Figure 7.2: IoT-Based Remote Patient Monitoring System

7.3 Activity Monitoring

Activity monitoring (Human Activity Recognition – HAR) uses sensors to identify what a person is doing, such as walking, running, sitting, sleeping or falling, and to measure fitness and health metrics.

System components

- **Sensors:** accelerometer, gyroscope, magnetometer (together called an IMU), heart-rate sensor, GPS, pressure sensor; camera or radar in ambient systems.
- **Devices:** smart watches, fitness bands (Fitbit, Mi Band, Apple Watch), smartphones, smart shoes, smart-home motion sensors.
- **Processing:** signal pre-processing → feature extraction (mean, variance, FFT) → classification using ML models (decision tree, SVM, random forest, CNN/LSTM).
- **Cloud/edge:** stores long-term history; edge processing on the wearable allows immediate fall detection.

Applications

- Fitness tracking – steps, calories, distance, sleep quality.
- Elderly monitoring and **fall detection** with automatic alerts.
- Rehabilitation and physiotherapy monitoring.
- Sports performance analysis.
- Worker safety and ergonomics in industry; smart-home occupancy and routine detection.
- Mental-health and lifestyle insights.

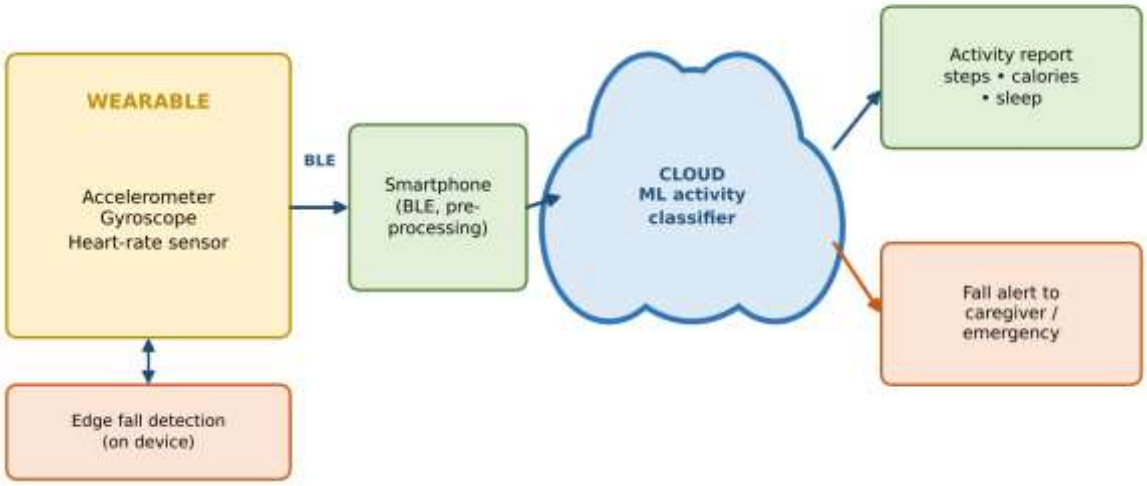


Figure 7.3: Wearable-Based Activity Monitoring and Recognition System

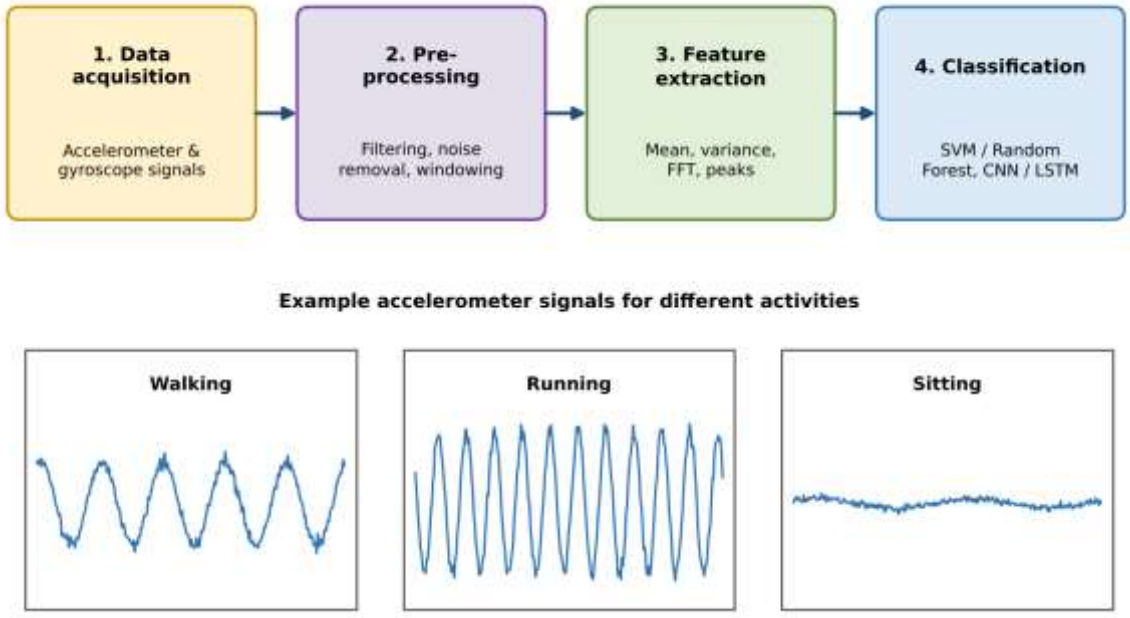


Figure 7.4: Activity Recognition Pipeline (Data Acquisition → Pre-processing → Feature Extraction → Classification)

8. Quick Revision Summary

Topic	Key points to remember
Cloud computing	On-demand, pay-as-you-go, elastic; IaaS/PaaS/SaaS; public/private/hybrid/community
Cloud storage	Object (S3), block (EBS), file (EFS), SQL, NoSQL / time-series databases
Edge computing	Processing at or near the data source; lowest latency; limited resources
Fog computing	Intermediate layer between edge and cloud (Cisco); local analytics and coordination

Topic	Key points to remember
Web servers	Apache, Nginx, Node.js; HTTP/REST, MQTT, CoAP; embedded vs cloud
AWS IoT	Device Gateway, Message Broker, Registry, Shadow, Rules Engine, Greengrass
Azure IoT	IoT Hub, DPS, Device Twin, IoT Edge, Stream Analytics, Digital Twins, Power BI
Sensor-Cloud	WSN + cloud; sensor virtualisation; Sensing as a Service
Analytics	Descriptive, diagnostic, predictive, prescriptive; batch vs stream; ML
Visualization	Line charts, gauges, heat maps, dashboards; Grafana, Power BI, ThingSpeak
Use cases	Smart city, smart home, connected vehicles (V2X), smart grid (AMI), IIoT (predictive maintenance)
Case studies	Agriculture (smart irrigation), healthcare (RPM), activity monitoring (HAR, fall detection)